

FACTORY LABS

Privacy on Octra

Ten Approaches and Where Each One Stopped

ABSTRACT

This report records every method by which we attempted to conceal trade amounts and participants in an automated market maker built on the Octra chain, and states precisely what stopped each one. Ten approaches were taken to a live test on the public devnet between July and September 2026. Four were stopped by platform limits that no amount of application design can work around; two work but are impractical at the rate a market requires; one works and is elegant but removes no blocker; one is written and formally verified but not yet deployed; two work and form the system that now runs. Every figure below was taken from confirmed devnet transactions and from the node source, not from estimation. The report closes with four specific platform changes, each of which would remove a named blocker.

Prepared for the Octra core team

Network under test: devnet.octrascan.io

Period of measurement: July to September 2026

Final measurements: 19 September 2026

Issued: 24 September 2026

Contents

1	Objective	
2	Method	
3	Approaches	
3.1	Encrypted balances held inside a Circle	<i>blocked</i>
3.2	A Circle owned by another Circle	<i>works</i>
3.3	A Circle holding a key of its own	<i>blocked</i>
3.4	Proof verification inside a program	<i>blocked</i>
3.5	A bridge from Circle to program	<i>blocked</i>
3.6	The chain's own stealth transfer	<i>narrow</i>
3.7	Fixed denominations carried by a relay	<i>impractical</i>
3.8	Tokens addressed to a key, not an address	<i>not deployed</i>
3.9	Ratios in place of amounts	<i>works</i>
3.10	A k-of-n committee	<i>works, but is trust</i>
4	Summary of blockers	
5	Platform changes that would remove them	
6	What was built, and what it conceals	
7	Conclusion	
A	Appendix A. Measurement reference	

Tables

Table 1. Summary of the nine approaches and their blockers.

Table 2. Reference measurements taken on devnet.

1 Objective

The objective did not change across the period. We wanted an exchange of two assets in which an outside observer, holding nothing but the public chain, cannot learn **the amount** of a trade and cannot learn **who** made it. Everything else, including throughput, fees and convenience, was treated as secondary to those two properties.

We did not set out to invent new cryptography. The premise throughout was that Oetra already ships the primitives a confidential exchange needs, and that the work was one of assembly. That premise turned out to be half right, and the half that failed is the subject of this report.

2 Method

Each approach was taken to a live test on the public devnet rather than to a local harness. A claim appears below only if a confirmed transaction or a node response supports it. Where a limit is stated as a number, that number was either read back from a transaction or located in the node source and then confirmed by a transaction that hit it.

Guards are reported as rejections. A guard that has not been made to fire on chain is not treated as proven, and several claims in earlier drafts of our own notes were withdrawn on exactly that ground.

Three contracts carry the final system and are live on devnet: the ledger Circle, the custody vault and a test token.

Ledger Circle	octcKtKPj7FsTRKifUpsurZjw4it2qJ3rWDBUgJDJSroc
Custody vault	oct6nni43s9dvHbeFFYGNgi58L9xS4bqikrjbnditjJsZRu
Test token	oct39N3ZexgkWzqPYZ1B2nqRYTM3qCdbZ9Eg7sHQBtB4Uyy

3 Approaches

The nine approaches are given in the order they were attempted. Each carries its intent, what was verified on chain, and the blocker that ended it or the limitation it left behind.

3.1 Encrypted balances held inside a Circle

Status. Blocked by a platform limit.

Intent

Every user balance and both pool reserves are held as ciphertexts under a single key. A trade is a homomorphic addition and subtraction performed inside the Circle, with no decryption at any point. This is the textbook shape of a confidential pool and was the first thing we built.

Verified on chain

The homomorphic primitives themselves work. Loading a public key, adding two ciphertexts and verifying a bound proof were each confirmed by real devnet transactions, including inside state-changing calls.

Fresh ciphertext	57,568 chars
The same value after one addition	115,052 chars
Maximum size of one Circle record	65,536
Two chained additions, read call, limit 2,000,000,000 effort	fuel exhausted

Blocker

A ciphertext produced by an operation can be neither stored nor fed into a further operation. This is not an inconvenience that a different data layout would avoid: a balance that changes *is by definition* the result of an operation applied to the previous balance. A confidential accumulator therefore cannot exist. Note that the second measurement below was taken in a read call, which is granted an effort limit two thousand times larger than a state-changing call; the operation still exhausted it.

3.2 A Circle owned by another Circle

Status. Works as designed; does not remove the blocker above.

Intent

Ciphertexts can only be combined under a common key, so a shared pool needs one key, and whoever owns that key can read every balance in the pool. The idea was to make the owner not a person but another Circle. Circles cannot call one another, so the condition under which decryption is permitted can never be satisfied by anybody.

Verified on chain

Confirmed by direct comparison. With a human owner, decryption succeeds. With a Circle as owner the node answers capability denied. Addition, subtraction and proof verification continue to work in both configurations.

Blocker

None of its own. The key is genuinely dead for every party, which is an elegant result, but the growth limit from 3.1 is untouched and the pool still cannot keep a running balance.

3.3 A Circle holding a key of its own

Status. Blocked by an unfinished code path in the node.

Intent

If a Circle can decrypt internally, the growth limit disappears completely. The Circle decrypts a balance, performs ordinary integer arithmetic, encrypts the result afresh and stores it. A fresh ciphertext always fits, so size never compounds.

Verified on chain

The Circle host interface exposes encryption and decryption, and, unlike every other homomorphic call it offers, neither takes a public key argument. That strongly implies a key supplied from within. We deployed a probe and asked the chain directly.

Circle's own public key	hfhe pubkey unavailable
Encryption	capability denied
Decryption	capability denied
Policy setting at the time of the test	encrypt: any_registered

Blocker

The denial does not come from policy. In the Circle under test, encryption was **permitted** to any registered caller, and the node refused anyway. The cause is visible in the node source: the field `circle_hfhe_key_id` is declared and immediately defaulted to empty, and no code path anywhere in the tree assigns it. The lifecycle operation `circle_key_grant` records a validity window for a key identifier but does not create key material. The plumbing is laid; the last joint is missing.

3.4 Proof verification inside a program

Status. Blocked by a withheld capability.

Intent

A Circle can verify proofs but holds no money. A program holds money. If a program could judge a proof itself, no intermediary would be needed between the statement that something was verified and the payment that follows from it.

Verified on chain

A probe program was deployed and interrogated step by step, both through read calls and through state-changing transactions.

Load a public key by address	permitted
Deserialise a ciphertext	permitted
Add two ciphertexts	permitted
Produce a commitment	permitted
Verify a zero proof	reverts
Verify a bound proof	reverts

Blocker

The refusal is identical in a read call and in a transaction, so it is not the historical view-only restriction. Nor is it cost: in the current build a verification is priced at 50,000 effort against a 1,000,000 limit for a state-changing program call, which fits with room to spare. The capability is simply withheld. The node grants a program almost the whole homomorphic surface and removes exactly one operation from it, and that one operation is the one on which a trustless bridge depends.

3.5 A bridge from Circle to program

Status. Blocked in three independent ways.

Intent

Given that a Circle can verify and a program can pay, the Circle should be able to tell the program that a condition holds, by a call, a signature, a receipt, or any other attestation the program could check.

Verified on chain

All three routes were examined against the node source and the Circle host interface, and the third was settled by enumerating the interface in full.

Blocker

A Circle cannot call a program. A program cannot read Circle state. A Circle cannot sign anything: the host interface exposes one hundred and seventeen calls and not one of them produces a signature. Node-issued proof receipts are likewise invisible to programs. A human being therefore stands between verification and payment, and there is no mechanism with which to remove them. This is the central structural finding of the report: **the half of the system that can verify and the half that can pay are deliberately separated, and nothing connects them.**

3.6 The chain's own stealth transfer

Status. Works exactly as advertised, within a narrow quota.

Intent

Oetra ships a native confidential transfer. If it performs, nothing needs to be built at all for the deposit and withdrawal path.

Verified on chain

A confirmed stealth transaction discloses nothing of substance. The sender field is a dash, the recipient reads *stealth*, the amount is zero and the message is empty. Only the fee, the epoch and the timestamp remain. This is the node's own rendering, not a client-side omission.

Time to build one transfer	8 to 15 minutes
Stealth operations per epoch, whole network	4
Confidential operations per epoch, whole network	8
Of three submitted concurrently	1 expired

Blocker

For a deposit made once, a quarter of an hour of computation is tolerable. For trading it is not. Four stealth operations per epoch across the entire network is a queue rather than a limit, and a transaction that misses its slot expires rather than waits. Of three submitted together, one was dropped with TTL exceeded.

3.7 Fixed denominations carried by a relayer

Status. Works; rejected as unusable.

Intent

Everyone trades in identical units, so that no trade can be told apart from another by its size, and a relayar submits the transaction on the user's behalf so that the chain records the relayar's address instead of the user's.

Verified on chain

The relayar half works and is proven. An unrelated wallet submitted a signed instruction on a user's behalf; it was accepted, and the user's own address never appeared on chain. The Circle checks the signature carried in the arguments, not the identity of the sender.

Blocker

The denomination half is the problem. A user cannot trade an arbitrary amount, only a multiple of the unit, and a larger trade means proportionally more transactions and proportionally more time. This was rejected on product grounds, correctly: a market in which one cannot name one's own size is not a market.

3.8 Tokens addressed to a key, not an address

Status. Works in principle. The primitive is proven on chain; the contract is written and formally verified but has not been deployed.

Intent

A sender escrows tokens against the recipient's **public key** rather than against an address. The recipient, at a time of their own choosing, signs a statement naming a payout address and hands that signature to anybody at all. Any party can carry it to the chain; the contract checks the signature itself and pays the address named inside the signed message. The recipient therefore never appears on chain until they decide to, and the address they nominate can be a fresh one with no history.

Verified on chain

The primitive the scheme rests on was confirmed separately. Ed25519 verification works inside a state-changing call, not only in a read, and costs 2,000 effort against a 1,000,000 limit. A probe contract accepted a correct signature and rejected a message that had been altered, both by confirmed devnet transactions. The claims contract itself compiles to 3,273 bytes and passes the node's formal verification with no errors and no warnings.

Ed25519 check inside a state-changing call	works
Effort cost of one check	2,000
Correct signature	accepted
Altered message	rejected
Contract size	3,273 bytes
Node formal verification	safe, 0 errors
Deployed	no

Blocker

It conceals the recipient, not the amount. Both the escrow and the claim carry the figure in clear text. What it does buy is worth stating precisely: the carrier holds no power, because the payout address is bound inside the signed message and can be neither substituted nor forged, and withholding the signature is pointless since the recipient can hand it to somebody else or submit it themselves. Combined with 3.7 it conceals both ends of a transfer, at the cost of fixed denominations. The contract was left undeployed when the work moved to 3.9, and it remains a valid building block rather than a closed question.

3.9 Ratios in place of amounts

Status. Works. This is what now runs.

Intent

Abandon homomorphic encryption for this purpose entirely. The node does not serve Circle storage to anybody except the owner, so balances can be held as ordinary integers and still be unreadable. Instructions then name a **ratio** rather than an amount: *trade two hundred and fifty thousandths of my A*. The ratio is public, the balance it applies to is not, and therefore the amount is not.

Verified on chain

Storage reads, full storage dumps and unauthorised view calls are all refused to a stranger holding a freshly generated key. Each user reads the Circle under their own signature and receives only their own figures, and the Circle sees who is asking and answers that party alone. Pool reserves are withheld too, so the amount cannot be recovered from price movement either. Constant-product arithmetic was checked against the reserves and agreed to the unit.

Homomorphic operations used	none
One-operation-per-call limit	does not apply
Ciphertext growth	does not apply
Per-epoch confidential quota	does not apply
A trade costs	one ordinary transaction

Blocker

What remains visible is a pseudonym for the internal account, the direction of the trade, the ratio, the timing, and the amounts at the deposit and withdrawal boundary, where value genuinely crosses the chain. The pseudonym links a user's own actions to each other in the way a reused Bitcoin address does. It is removable by taking a fresh internal account per deposit; the boundary amounts are not removable without approach 3.6, or narrowed by 3.8.

3.10 A k-of-n committee

Status. Works. It is narrowed trust, not a proof.

Intent

Since a Circle cannot tell a program that a condition holds, several independent key holders say so instead, and the program checks their signatures itself. Signature verification is cheap enough that the committee can be large: at 2,000 effort against a 1,000,000 limit, roughly four hundred checks fit in a single call.

Verified on chain

Seven guards were each made to fire on chain. Two signatures are insufficient. A signature over a different message does not match, so neither the recipient nor the amount can be substituted. Three valid signatures suffice. Payment before the delay elapses is refused. A second payment against the same order is refused. The per-window ceiling refuses the payment without discarding the order, which then waits for the next window. Deposits are gated the same way and a deposit identifier cannot be credited twice.

One practical convenience is worth recording: a committee member needs exactly one key. The Oetra address with which they read the Circle is derived from the same public key whose raw bytes the vault uses to check their signature.

Blocker

This is trust with its radius reduced, and it should not be described as anything else. A committee that colludes at the threshold can still misreport the ledger, because the program cannot read the Circle and has no way to check. The guards bound how much can be removed and how quickly; they do not prevent collusion. This is precisely the model on which RenVM operated and ultimately failed.

4 Summary of blockers

Table 1 collapses the preceding section. Four entries are hard platform limits, two are rate or usability limits, and three are the components of the system that now runs.

§	Approach	What stopped it
3.1	Encrypted balances	A result can be neither stored nor reused as an input
3.2	Circle-owned key	Works, but leaves 3.1 in place
3.3	Circle's own key	The key field is never populated by any code path
3.4	Verification in a program	The capability is withheld from programs
3.5	Circle to program bridge	No call, no read, no signature
3.6	Stealth transfer	Quarter of an hour each, four per epoch network-wide
3.7	Fixed denominations	An arbitrary amount cannot be traded
3.8	Tokens sent to a key	Conceals the recipient, not the amount; not deployed
3.9	Ratios in place of amounts	Leaves a pseudonym and the boundary amounts
3.10	Committee	Trust rather than proof

Table 1. The nine approaches and their blockers.

5 Platform changes that would remove them

Four changes are set out below. They are not equivalent, and the first two are alternatives to one another: either would make a confidential pool possible with no trusted party at all. Each is annotated with the sections it would unblock.

5.1 Grant Circles the ciphertext compaction primitive

The node's own library contains the operation that compacts a grown ciphertext. It requires a key derived from the secret half, and it does not appear anywhere in the Circle host interface. Exposing it to Circles revives approach 3.1 in full, and with it the ordinary confidential pool that most people expect when they hear the phrase.

Unblocks 3.1

5.2 Or populate the Circle key identifier

The field and the entire apparatus around it are already written: policy modes, lifecycle transitions, capability gating. What is missing is any code that assigns the identifier. Once assigned, a Circle computes in the clear internally and stores a fresh ciphertext, and ciphertext growth stops being a constraint at all.

Unblocks 3.1 and 3.3

5.3 Restore proof verification to programs

The view-only restriction has already been lifted in the current build, and the cost is 50,000 effort against a 1,000,000 limit. Only the capability grant is outstanding. With it, a money-holding program adjudicates for itself and the committee of section 3.10 becomes unnecessary, which removes the only trusted party in the present design.

Unblocks 3.4, 3.5 and 3.10

5.4 Raise the stealth quota

Four operations per epoch across the whole network is a queue in which transactions expire rather than wait. An order of magnitude would make the native stealth transfer usable for the deposit and withdrawal boundary of a pool, which is where the remaining disclosure in the present design sits.

Unblocks 3.6

6 What was built, and what it conceals

The system that runs today rests on approaches 3.9 and 3.10. A Circle keeps the ledger and the pool in ordinary integers that nobody outside can read. Instructions are expressed as ratios, so no amount appears in any transaction. Each participant reads their own position under their own signature. A relay submits the transaction, so the chain records the relay. Custody sits in a program that pays only on three signatures out of five, after a delay, and within a per-window ceiling.

Concealed, and verified as concealed: every balance, every liquidity share, every trade amount, and the association between a participant and their chain wallet.

Not concealed: the pseudonym of the internal account, the direction of each trade, its timing, and the amounts crossing the deposit and withdrawal boundary.

The honest summary is that privacy was obtained somewhere other than where we were looking for it. It did not come from the confidential arithmetic, which ran into sections 3.1 and 3.3. It came from two plain facts: Circle storage is not served to outsiders, and an instruction can name a ratio instead of a number.

7 Conclusion

Oetra's confidential arithmetic works. It adds, subtracts, multiplies a ciphertext by a public scalar, and verifies proofs, and we confirmed each of those on the live chain. There is one thing it cannot do, and the whole difficulty follows from it: **it cannot accumulate**. A result can be neither stored nor used as the input to the next operation. Every balance is an accumulator, so no balance can be confidential.

Alongside that sits a second and quite separate finding. The capability to verify a proof and the capability to move value are held by different execution surfaces, and no mechanism connects them. A Circle can verify and cannot pay. A program can pay and is not permitted to verify. Neither can call, read or attest to the other. Consequently every design that reaches a confidential result must still hand that result to a trusted party in order to act on it.

Both findings are properties of the platform, not of our application, and both are addressed by the changes in section 5. Until then, a confidential exchange on Oetra is achievable only in the reduced form described in section 6: amounts genuinely hidden, participants pseudonymous, and a committee holding custody at the boundary.

Appendix A Measurement reference

All figures below were taken on devnet.octrascan.io. Sizes are in characters as returned by the node; effort figures are the units used by the virtual machine.

Fresh ciphertext	57,568
Ciphertext after one homomorphic addition	115,052
Maximum size of one Circle record	65,536
Circle code, stable storage and asset limits, each	33,554,432
Effort limit, state-changing program call	1,000,000
Effort limit, read call	2,000,000,000
Effort cost, zero or bound proof verification	50,000
Effort cost, Ed25519 signature check	2,000
Circle call fuel ceiling	20,000,000
Ceiling required for a call that checks a signature	6,000,000
Ceiling sufficient for a call that does not	1,000,000
Stealth operations per epoch, whole network	4
Confidential operations per epoch, whole network	8
Time to build one stealth transfer	8 to 15 min
Circle host interface calls, total	117
Of which produce a signature	0

Table 2. Reference measurements taken on devnet.

A note on fees, since it cost us a good deal of test currency to learn. A Circle call is charged its declared ceiling **in full** when it succeeds, and nothing at all when it fails. A generous ceiling is therefore not free: it is the price. Twelve calls declared at twenty coins consumed two hundred and thirty.